Navigating Runtime Dynamism in Semantic Query Serving

Hengrui Zhang
Tsinghua University
Beijing, China
zhanghen22@mails.tsinghua.edu.cn

Xupeng Miao
Peking University
Beijing, China
xupeng.miao@pku.edu.cn

ABSTRACT

LLM-powered semantic queries increasingly combine relational processing with multiple LLM-based operators over unstructured data. Efficiently serving such queries is challenging due to runtime dynamism at two coupled levels. Across queries, dynamic arrivals and heterogeneous resource demands continuously change scheduling priorities. Within a query, heterogeneous characteristics across fine-grained input batches cause operator costs and selectivities to vary throughout execution, making a statically optimized plan unreliable. These two forms of dynamism create a circular dependency: inter-query scheduling requires estimates of remaining LLM work, while the remaining work depends on runtime plan choices whose costs become observable only during execution.

We present SeqoFlow, a runtime framework that jointly adapts query plans and schedules concurrent semantic queries. SeqoFlow enumerates equivalence-preserving plan alternatives under operator dependencies and decomposes queries into fine-grained units, enabling safe plan adaptation and query preemption at batch boundaries. To support both decisions without exhaustively profiling candidate plans on the target LLM, SeqoFlow uses lightweight speculative estimation to capture runtime costs. These estimates jointly guide plan adaptation and inter-query scheduling, closing the loop between query optimization and LLM inference. Experiments across diverse query structures, data distributions, and arrival rates show that SeqoFlow substantially reduces end-to-end latency compared to existing semantic query systems, adaptive query processing techniques, and conventional scheduling policies.

Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Seurgul/SeqoFlow/tree/main.

1 INTRODUCTION

LLM-powered data analytics has emerged as a new paradigm, attracting growing interest from both academia and industry [3, 4, 7, 25, 37, 38, 44]. By incorporating LLM-powered semantic operators into analytical queries, these systems allow users to express ad hoc intents such as semantic filtering, information extraction, entity matching, ranking, and summarization. Modern data platforms are consequently evolving from monolithic query engines into hybrid systems that tightly couple relational query execution with GPU-backed LLM inference.

Efficiently serving concurrent semantic queries, however, is complicated by pervasive ***runtime dynamism*** at two levels. At the inter-query level, online query arrivals, completions, and heterogeneous LLM demands continuously alter which query should be served next. At the intra-query level, heterogeneous characteristics across fine-grained input batches, together with content-dependent model behavior, cause operator costs and selectivities to vary throughout query execution.

For example, the following semantic query assesses antibiotic usage in infectious diseases among Asian pediatric departments by combining two medical research paper datasets [18]. At runtime, the selectivity of natural-language predicates can vary substantially, making downstream cost estimation unreliable. Moreover, several operators consume outputs generated by upstream LLM operators, introducing cost correlations that are difficult to predict and detect. Executing such queries with a fixed run-to-completion plan can therefore lead to suboptimal latency and unnecessarily delay other concurrent queries sharing the same LLM inference resources.

```
SELECT t2.antibio
FROM (SELECT fulltext_1 FROM table_1
      WHERE sem_filter('{affiliations_1} is an Asian institution.')
        AND sem_filter('{abstract_1} comes from Pediatrics.')) t1
sem_join
     (SELECT sem_map('Assess antibiotic use in {drugs_2}') antibio,
             sem_map('List the drugs studied in {abstract_2}') drugs_2
      FROM table_2
      WHERE sem_filter('{abstract_2} studies infectious diseases.')) t2
ON '{drugs_2} is studied in {fulltext_1}';
```

Crucially, these two forms of runtime dynamism cannot be addressed independently. Inter-query scheduling requires an estimate of each active query's remaining LLM work. Existing query scheduling techniques typically assume that query execution costs can be estimated or provided before execution and optimize scheduling decisions based on such information [23]. However, the remaining work of a semantic query depends on the execution plan selected for its unprocessed input batches. Changing the order or placement of semantic operators may substantially alter intermediate cardinalities and the number and token volume of subsequent LLM invocations. Selecting an efficient plan, in turn, requires operator costs and selectivities that are difficult to predict accurately using static statistics and may become observable only during execution. A runtime plan change therefore revises the query's remaining-work estimate and may alter its scheduling priority relative to other active queries. Subsequent execution exposes new runtime observations that may trigger further plan and scheduling decisions. This creates a *circular dependency* among runtime observation, intra-query plan adaptation, remaining-work estimation, and inter-query scheduling.

Existing techniques address individual parts of this cycle but do not close it end to end. Research on expensive predicates and UDFs studies cost-aware operator placement, while adaptive query processing revises execution decisions using runtime statistics. Recent LLM-powered data systems provide logical and physical optimizations for LLM-powered operators [38, 44, 63]. Separately, database and LLM serving systems support query preemption [23], request scheduling [33], and GPU resource management [26]. However,

query optimizers generally select a plan for each query in isolation, without considering how that choice changes the query's remaining work relative to other active queries, whereas LLM schedulers treat inference requests or entire queries as scheduling units without reasoning about alternative query plans. Applying these techniques independently leaves the scheduler with stale remaining-work estimates whenever a query plan changes, and drives each query to commit to a plan before its operator costs become observable.

In this paper, we argue that navigating runtime dynamism in semantic query serving requires coordinating two individually distinct but mutually dependent decisions: *adapting query plans to intra-query data heterogeneity and scheduling concurrent queries under inter-query workload dynamics*. This coordination raises two key challenges.

C1: Scheduling under Inter-Query Dynamism. As semantic queries arrive, complete, or expose updated estimates of their remaining work, their relative scheduling priorities continuously change. Run-to-completion execution can cause severe head-of-line blocking, whereas scheduling individual LLM requests discards the query structure needed to estimate downstream work. The system therefore needs a scheduling unit that supports timely preemption while retaining sufficient execution context. This raises the first question: *How can the system efficiently switch among concurrent semantic queries and schedule them according to dynamically changing estimates of their remaining work?*

C2: Plan Adaptation under Intra-Query Dynamism. Fine-grained input batches within a query may exhibit heterogeneous characteristics and execution behavior, causing operator costs and selectivities to vary throughout execution. A statically selected plan may therefore not remain efficient for every batch. However, runtime adaptation must account for input batches that have already passed through parts of the plan, as well as attribute dependencies, operator semantics, and blocking boundaries. This raises the second question: *How can the system safely adapt the execution plan for a query's remaining input batches while preserving equivalence and operator dependencies?*

To address these challenges, we present SeqoFlow, a unified runtime framework that *jointly* adapts query plans and schedules concurrent semantic queries. SeqoFlow constructs equivalence-preserving plan alternatives under explicit operator dependencies and decomposes query execution into batch-level *Subquery Execution Units* (SEUs). SEUs provide safe boundaries at which SeqoFlow can incorporate runtime statistics, select a compatible plan for the remaining input batches, preempt an active query, and switch to another query without discarding completed work. SeqoFlow coordinates plan adaptation and inter-query scheduling through lightweight speculative profiling. It profiles a representative subquery using a smaller language model, collects operator-level selectivity and token-processing costs, and propagates these statistics across compatible plan alternatives to estimate their remaining execution costs. The same estimates guide the global scheduler in reprioritizing active queries, thereby closing the runtime optimization–scheduling loop without exhaustively profiling every candidate plan on the target LLM. We implement SeqoFlow as an end-to-end semantic query serving system integrated with a GPU-based LLM backend. Experiments across diverse query structures, batch-level data characteristics, operator dependencies, and arrival rates demonstrate that SeqoFlow substantially reduces end-to-end latency compared with existing semantic query systems, adaptive query processing techniques, and conventional scheduling policies.

In summary, this paper makes the following contributions:

- We propose SeqoFlow, a unified framework to support efficient serving of semantic queries under runtime dynamism.
- We present a runtime adaptation mechanism to capture and select better plans while preserving logical equivalence.
- We introduce a fine-grained, structure-aware execution abstraction that exposes safe preemption boundaries and enables dynamic scheduling across concurrent semantic queries.

2 PRELIMINARY AND RELATED WORK

2.1 Optimizing LLM-powered Data Systems

Recent data systems (e.g., [38, 44]) increasingly rely on LLMs as the primary workhorse over unstructured data. These systems provide declarative interfaces that allow users to program using semantic operators or more general LLM pipelines [52]. In practice, a semantic query is executed by applying each semantic operator to its input data through one or more LLM calls.

Previous work has mainly focused on physical optimization of semantic queries, including accelerating individual operators [27, 28, 44, 48], prompt rewriting [52], and extending the Cascades optimizer [49]. Regarding logical optimizations, [37–39, 54] directly apply rule-based rewrites, such as operator pull-up, while [63] prompt LLMs to perform logical rewrites guided by human-written heuristics. Overall, this literature reflects a shift from treating LLM calls as black-box functions toward principled query models and optimizers for LLM-augmented analytics. Notably, existing approaches optimize individual, shared-nothing workloads in isolation, while production environment faces more limitations from contentions for bounded resources [23, 56], including LLM inference service. A unified abstraction for expressive semantic querying that coordinates query optimization with preemptive, query-level scheduling remains open.

2.2 General Serving and Scheduling

Existing LLM serving systems adopt general-purpose scheduling policies such as FCFS [33] or MLFQ [58], which treat each incoming request as a monolithic unit. Some systems improve scheduling by leveraging techniques like chunked prefill [8], disaggregation [45], preemption [58], migration [53], and output length prediction [46].

While these approaches are effective for homogeneous inference workloads, they do not directly support semantic queries. A semantic query is a structured operator graph whose execution cost depends on operator costs, ordering, and intermediate data sizes. From a scheduling perspective, this introduces two challenges. First, policies such as SRTF require reliable estimates of remaining query time, yet semantic query costs are content-dependent and evolve during execution. Second, request-level schedulers also ignore query progress and plan structure. Effective scheduling must therefore account for both query-level and operator-level costs.

2.3 Logical Optimizations in Relational DBMS

Beyond the techniques described in Section 2.1, logical query optimization has a rich history, particularly for expensive operators and UDFs that invalidate simple optimization heuristics. Early work [14, 22, 35] studies the cost-based placement of predicates and joins, showing that rules such as eager predicate evaluation can become suboptimal when operator costs are non-negligible. Other work [41, 42] develops dynamic-programming (DP) algorithms that identify optimal partial plans under known cost and selectivity estimates. These approaches provide promising correctness within their respective settings, but rely on predefined operator semantics and sufficiently stable cost information. They therefore do not directly accommodate fluctuating costs, new query language extensions [52, 59], and latent correlations among operators [31, 36, 61].

Another line of work studies adaptive query processing, which revises execution decisions using observations collected at runtime. For example, Eddies [9, 26] are adopted in ML-powered data systems. However, Eddies do not provide a general mechanism for enforcing expressive dependencies among operators, and they require sophisticated implementations and support limited semantics (e.g., blocking operators such as aggregation cannot be easily translated into Eddies' scheme). Meanwhile, query re-optimization [24, 62] mainly considers adapting to a new plan after partial execution of individual queries by re-running the optimization algorithms. Therefore, it cannot directly address the highly dynamic characteristics in analyzing unstructured data. Our goal is to combine the logical optimization's equivalence preserving with the runtime responsiveness of adaptive query processing.

3 OVERVIEW

3.1 Workload Formulation

We consider an online scenario where analytical queries arrive ad hoc at the system frontend. Each query request q can be formulated as a set of logical operators comprising both traditional relational operators (e.g., filters, joins) and LLM-based semantic operators. Upon arrival, the frontend queues these requests and dispatches them to a centralized backend execution engine, which then interacts with both structured and unstructured data.

Established works that integrate ML/LLM inference within relational data systems [25, 30, 44] as well as production standards [2, 5] fetch and process data in fixed-size batches (i.e., multiple rows/tuples) to improve hardware utilization. Batches naturally discretize continuous query execution into a sequence of measurable states. Formally, we define the execution state at any given moment as $state = (q, \{op_i\}, B_j)$, representing the query served q, the active physical operators $\{op_i\}$, and the current data batch B_j. The continuous interplay of these three components creates a dynamic and complex execution landscape. Because the computational cost of semantic operators depends heavily on the semantic content and input distribution of B_j, the optimal execution strategy can shift drastically mid-flight. From our observations, a query plan applied to $\{op_i\}$ that was optimal for an early batch may become a severe bottleneck for subsequent batches, requiring dynamic plan mutation. Similarly, the execution time of queries q fluctuates as batches are processed, demanding dynamic scheduler adjustments.

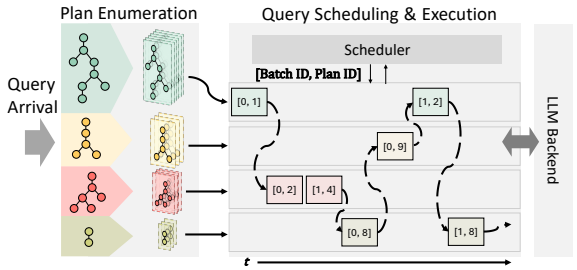


Figure 1: System Architecture – SeqoFlow consists of two stages: A plan enumeration stage that captures equivalent logical plans, and a runtime that schedules and executes batch-level jobs and dynamically selects plan alternatives for batch-level execution.

Despite such volatility, current frameworks fail to exploit the batch-level state granularity that co-optimize all the dimensions. Instead, they treat queries as monolithic tasks bound to a run-to-completion execution model. This motivates the need for a unified framework capable of dynamically estimating the cost of each transient *state*, unifying both mid-flight query optimization and fine-grained batch-level scheduling.

3.2 System Design

Figure 1 provides a high-level overview of SeqoFlow's architecture. The framework consists of two primary components: a static preprocessor that enumerates all equivalent physical plans for a given query q (§4), and a dynamic execution engine that orchestrates adaptive query processing and inter-query scheduling based on the instantaneous system *state* (§5). Upon query arrival, the pre-process generates and memoizes a comprehensive search space of equivalent query plans. This allows the runtime engine to rapidly retrieve and evaluate alternative execution strategies without re-parsing the query. During execution, the runtime processes queries in discrete *batch-level preemption units*. This fine-grained architecture enables the global scheduler to safely context-switch and prioritize queries based on dynamic cost metrics, resolving **C1**. Simultaneously, to address the intra-query data dynamism in **C2**, SeqoFlow leverages these units to continuously evaluate the pre-computed search space. By monitoring the current state, SeqoFlow dynamically migrates to the most cost-effective logical plan.

The effectiveness of SeqoFlow's co-design hinges on the ability to accurately estimate costs mid-flight without severely blocking query execution. To bridge this gap, we introduce a novel speculative cost estimation mechanism. By utilizing a lightweight Small Language Model (SLM) to profile transient batch latencies, the system captures real-time costs with high fidelity. This speculative feedback loop continuously guides the decisions in scheduling and plan adaptation with bounded computational overhead.

4 INTRA-QUERY PLAN ENUMERATION

The optimal execution order of semantic operators is not deterministic with simple rule-based heuristics. Execution performance is largely dependent on the physical cost of LLM-centric procedures and data characteristics, which are not known a priori and can vary

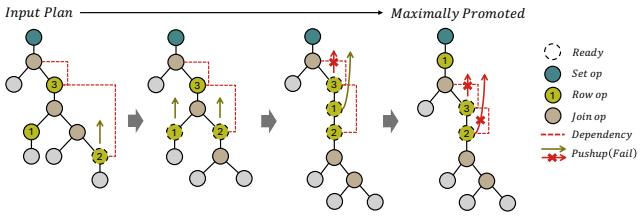


Figure 2: Operator Pushup Phase.

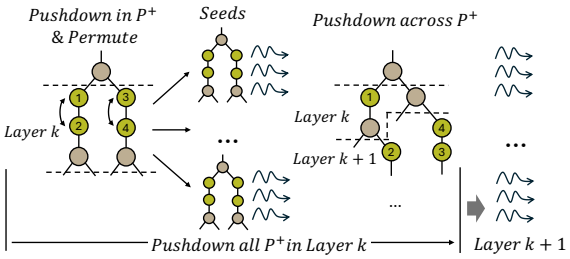


Figure 3: Operator Pushdown Phase.

substantially. This introduces significant challenges for transformation schemes applying the predefined set of rules, as they often fail to capture optimal plans falling outside the static patterns. Unlike prior systems [16, 39, 49, 52] that optimize queries by exploring quality–cost trade-offs, we restrict ourselves to lossless optimization and consider only plan transformations that preserve semantic equivalence[1]. In this section, we discuss the algorithms towards an enumeration of equivalent logical plans, enabling the discovery of the best candidate plan structures.

4.1 Problem Formulation

4.1.1 Logical Plans. In prevalent relational query models, a *logical plan* $T = \{op\}$ is a directed tree structure. Each node in the tree represents a logical operator op with at most two children, while edges denote the flow of data between operators. Leaf nodes correspond to `scan` operators over base relations (tables), fetching batches of data into physical processing. The root node produces the final output of the query. Binary operators (`join` and `sem_join`) are the only operators with two children, computing the product of two subtree relation[2]. Therefore, they serve as anchors to the plan topology space, which defines the unique tree structure and will not be modified during the transformation process when join reordering is not considered.

Definition 1. *For each join j, the source partition $P^+(j)$ consists of j and the operators above it up to the next join or the root. The sink partition $P^-(j)$ consists of j and the operators below it down to the next joins or scans. A plan without joins forms a single partition.*

The source and sink partitions act as the unit of transformations in the divide-and-conquer strategy for plan enumeration in §4.2.

4.1.2 Transformation. The optimizer enumerates equivalent plans by iteratively transforming from a seed plan T to a new plan T' through a step-by-step *swap* between parent and child operators, namely op_{parent} and op_{child}. However, to enforce lossless execution, certain swaps are prohibited as they violate the commutativity rules. Specifically, two rules are considered to determine commutativity:

Rule 1: Data Dependency If op_{parent} consumes a new attribute produced by op_{child}, the swap is not allowed as it would raise data dependency error.

Rule 2: Semantic Integrity Certain operator commutations fundamentally alter intermediate relations, violating the exact semantic equivalence of the query. For example, pushing a selection predicate upstream of an aggregation prematurely filters rows, resulting in an incomplete aggregate computation. To guarantee correctness in plan transformations, we categorize operators into three distinct classes: *row-wise* (e.g., filters, projections), *set-wise* (e.g., aggregations, sorts), and *joins*. Provided the data dependency constraints of Rule 1 are met, a swap can be applied safely between two consecutive row-wise operators, or a row-wise operator and a join, because these operations evaluate individual rows independently. To prevent semantic corruption, commutations involving any other class combinations are strictly prohibited.

It's noteworthy that the rules are sufficient rather than necessary, as commutations such as pushing a filter on a grouping or sort key also preserve equivalence but lie outside the enumerated space. Such cases are eliminated to simplify the algorithms' corner cases as it's hardly profitable to inflate a blocking operator's massive gathered input in favor of saving cost from a filter.

Equivalence Guarantee By strictly following the commutativity rules above, each swap can be considered as an equivalence-preserving operation. An admissible swap changes the order in which operators apply, and hence intermediate cardinalities and cost, but never the output bag. Formally, plans $T \equiv T'$ if they produce the same output bag for every valid input. A row-wise operator $\omega = (p, f)$ applies a predicate p and projection f: $\omega(R) = \{\{ f(t) \mid t \in R,\ p(t) \}\}$. Here R is an input bag. Let $\mathrm{Rd}(\omega)$ be the attributes read by p or f, and $\mathrm{Md}(\omega)$ the attributes created by f.

Lemma 1 (Swap Equivalence). *(i) Two row-wise operators ω_1, ω_2 commute if $\mathrm{Rd}(\omega_i) \cap \mathrm{Md}(\omega_j) = \varnothing$ for $i \neq j$ and $\mathrm{Md}(\omega_1) \cap \mathrm{Md}(\omega_2) = \varnothing$. (ii) A row-wise operator ω can move from above $R \bowtie_\theta S$ to the R branch if it reads only attributes of R and changes no attribute of S or any attribute used by the join predicate θ. The symmetric condition applies to S.*

Proof. In case (i), the conditions give $p_i(f_j(t)) = p_i(t)$ for $i \neq j$ and $f_1(f_2(t)) = f_2(f_1(t))$ for every row t. Both orders therefore retain exactly the same rows satisfying $p_1(t) \wedge p_2(t)$ and map each to $f_1(f_2(t)) = f_2(f_1(t))$. In case (ii), for rows $r \in R$ and $s \in S$, let $r \circ s$ concatenate r and s. The conditions give $p(r \circ s) = p(r)$, $f(r \circ s) = f(r) \circ s$, and $\theta(f(r) \circ s) = \theta(r \circ s)$, which preserve each joined pair and its output row. □

By transitivity of $T \equiv T'$, every plan reachable from the seed plan through a sequence of admissible swaps is equivalent to it, which underpins the correctness of the algorithms in §4.2. The cost of transformations and the size of the plan space are bounded as follows, with proofs in Appendix A [6]. Let N be the number of operators and J the number of joins. The hook length $h_F(op)$ counts op and its descendants in its constructed Hasse graph F.

[1]Note that such equivalence is defined with respect to the deterministic semantics of LLM operator intent, ignoring stochastic variation in LLM outputs.
[2]While we only discuss binary operators in this paper, the discussion can be extended to any n-ary operators without loss of generality.

Proposition 1 (Plan Space). *The number of plans reachable through admissible swaps is at most* $\frac{N!}{2^J \cdot \prod_{op} h_F(op)}$.

Proposition 2 (MECE). *Every plan memoized by Alg. 1 and Alg. 2 is equivalent to the seed plan and stored once. Without pruning, the memoized plans are* ***mutually exclusive and collectively exhaustive*** *(MECE) over the equivalence-preserving closure defined by rules in §4.1.2. With pruning, for every cost setting, they include a plan of minimum cost.*

4.2 Plan Enumeration

The plan enumeration strategy organizes the generation of different plans by first finding an initial state and then spawning equivalent variants from it recursively. We choose the *maximally promoted tree* as the initial state. It refers to the topology that all operators cannot be further swapped up with their parents. The process first leverages the **Operator Pushup** phase to reach this state, which is unique up to the order of commutative operators within each partition, then tries spawning equivalent plans by pushing down operators from the initial state, i.e. the **Operator Pushdown** phase. The procedure is summarized in Figure 2 and Figure 3.

Operator Pushup The goal and intuition of the pushup phase is to promote all row-wise operators to partitions of higher levels until they cannot be promoted anymore. The recursive procedure as present in Algorithm 1 and Figure 2 starts by promoting operators in the bottom-most sink partitions to the top. Each sink partition is a subtree rooted at join operators. The transformation takes each step by selecting a ready operator (denoted *active* in Alg. 1) from the children branches and swapping it up to above the root, or terminating with a failed swap due to commutativity constraints. This is orchestrated inside subroutine PushupInSinkPartition, determining whether all operators in the current partition can no longer be emitted to the parent partition above. If so, the transformed partition can be safely merged into the original plan (line 7 in Alg. 1), and the parent partition starts emitting its operators upward in the next recursion. Note that pushup mainly operates on partition granularity, and we do not count for the exact order of chained, commutative operators (e.g., the order of operators 2 and 3 in the final state of Figure 2) in the same partition. Different orderings of these chained operators are enumerated in the pushdown phase. The pushup phase is only used to arrive at a starting point for enumerating all admissible plans later. Therefore, no intermediate state needs to be cached.

Operator Pushdown After the pushup phase, we enumerate the full transitive set under Rules 1 and 2's equivalence guarantee by gradually pushing operators down the maximally promoted state. A source partition can be considered as a chain of operators with one join at the source (upstream). We assign each partition with its layer in top-down order. Algorithm 2 and Figure 3 describes a top-down transformation that pushes operators down to source partitions in lower layers. Operators migrate down (upstream) via swaps, with a memoizer capturing each unique intermediate state along the way.

Intuitively, partitions can spawn variants through two subroutines: **(1)** Reordering operators within the same partition, and **(2)** Pushing operators below the source join into lower partitions,

Algorithm 1 Operator Pushup

```
1:  Input: Plan T
2:  function Pushup(P)
3:      (P_l, P_r) ← GetChildrenSinkPartition(P)
4:      Pushup(P_l)
5:      Pushup(P_r)
6:      P' ← PushupInSinkPartition(P)
7:      T' ← Merge(T, P')
8:      return T'
9:  function PushupInSinkPartition(P)
10:     P' ← P
11:     active ← P.sink.children
12:     while active ≠ ∅ do
13:         for v ∈ active do
14:             if CanSwap(v.parent, v) then
15:                 P' ← Swap(v.parent, v)
16:                 UpdateDep(P')
17:         active ← GetActiveOps()
18:     return P'
```

namely PdInSrcPtt (Alg. 3) and PdAcrossSrcPtt (Alg. 4). The first subroutine permutes chained operators within each parallel layer-k partitions (e.g., in Figure 3, there exist two partitions in layer k with two chained operators in each). The Cartesian product of parallel partitions' variants is merged into the plan and memoized. These variants then seed the second subroutine, which records a new variant whenever an operator is pushed below a join. Again, the Cartesian product of these second-stage variants is merged and memoized. They become the starting plans in the next iteration on layer-$k+1$ partitions. During this stage, ApplyPruneHeuristic (line 26, Alg. 3) applies conventional rules, such as predicate pushdown of regular filter, to eliminate dominated candidates. A regular filter costs negligibly compared with an LLM call and never increases cardinality. Any candidate placing it above an LLM operator is therefore dominated by one placing it below, and pruning such candidates never removes the cheapest plan. After iterating over each layer top-down, Alg. 2 is completed with all operators opportunistically pushed-down in the lowest layer and all equivalent plans memoized, except dominated plans removed by pruning. Without pruning, the enumeration is mutually exclusive and collectively exhaustive (Appendix A [6]).

Running Example. For the query presented in §1, the two filters on `table_1` lie in the left sink partition of `sem_join`; operators above the join form its source partition. Pushup promotes eligible row-wise operators toward the join, but the map producing `drugs_2` cannot cross it because the join predicate reads that attribute. Pushdown enumerates both orders of the independent left filters and admissible placements of other row-wise operators. Every candidate keeps drug extraction before the join and before the map producing `antibio`.

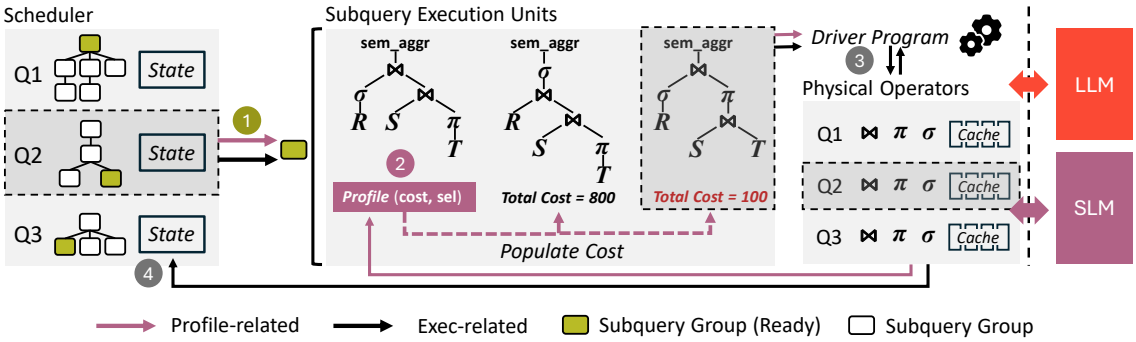


Figure 4: One Complete Iteration of SpecAQP – The workflow consists of 4 stages: 1. The Scheduler picks the next query to be executed and its current ready subquery group; 2. SEU profiles the subqueries to get costs; 3. SEU executes the cheapest subquery in the group; 4. Scheduler updates its internal states for the next iteration.

Algorithm 2 Operator Pushdown

Input: Plan T, Memoizer M
function PUSHDOWN(T, k)
  $\mathcal{P} \leftarrow$ GETALLKTHLAYERPARTITION(T, k)
  $seeds \leftarrow \varnothing$
  for $P_i \in \mathcal{P}$ **do**
    $seeds \leftarrow seeds \cup \{$PDINSRCPTT($P_i$)$\}$
  $seeds_2 \leftarrow \varnothing$
  for $comb \in$ CARTESIANPRODUCT($seeds$) **do**
    $T' \leftarrow$ MERGE($T, comb$)
    $M \leftarrow M \cup \{T'\}$
    $seeds_2 \leftarrow seeds_2 \cup \{comb\}$
  $plans_next \leftarrow \varnothing$
  for $comb \in seeds_2$ **do**
    for $comb'' \in \prod_{P'_i \in comb}$ PDACROSSSRCPTT(P'_i) **do**
      $T'' \leftarrow$ MERGE($T, comb''$)
      $plans_next \leftarrow plans_next \cup \{T''\}$
  for $T'' \in plans_next$ **do**
    PUSHDOWN($T'', k+1$)
  return M

Algorithm 3 Pushdown in Source Partition

function PUSHDOWNROWWISEOPS(P, op_type)
  UPDATEDEP(P)
  $operators \leftarrow$ GETOPSBOTTOMUP($P.src, op_type$)
  for $op \in operators$ **do**
    while CANSWAP($op, op.children$) **and** $op.children$ is not $\bowtie$ **do**
      UPDATEDEP(P)
      $P \leftarrow$ SWAP(op, op.children)
  return P
function FUSEDEPENDENCYGROUP(P)
  $dep_grps \leftarrow \varnothing$
  $op_ptr \leftarrow P.src.parent$
  $anchor \leftarrow op_ptr$
  $current_grp \leftarrow \{op_ptr\}$
  while $op_ptr \notin \{\bowtie$, stateful ops$\}$ **do**
    if CANSWAP($op_ptr, anchor$) **or** $op_ptr = anchor$ **then**
      $current_grp \leftarrow current_grp \cup \{op_ptr\}$
    else
      $dep_grps \leftarrow dep_grps \cup \{current_grp\}$
      $anchor \leftarrow op_ptr$
      $current_grp \leftarrow \{op_ptr\}$
    $op_ptr \leftarrow op_ptr.parent$
  return dep_grps
function PDINSRCPTT(Partition P)
  $P \leftarrow$ PUSHDOWNROWWISEOPS($P, \{\sigma, \pi, \ldots\}$)
  $dep_grps \leftarrow$ FUSEDEPENDENCYGROUP(P)
  $dep_grps \leftarrow$ APPLYPRUNEHEURISTIC(dep_grps)
  $seeds \leftarrow \cup$APPLYPERMUTE(dep_grps)
  return $seeds$

5 INTER-QUERY SERVING

This section presents the serving infrastructure supporting online arrivals of semantic queries. The middleware exposes a backend-independent execution interface, allowing it to integrate with different inference engines and storage systems. Backend independence concerns the interface rather than the absence of execution feedback. The feedback is obtained at batch boundaries (i.e., execution costs) without relying on engine-specific internals. As discussed in §1, the effectiveness hinges on overcoming both **C1** and **C2**. First, it requires support for low-level preemption of in-flight queries, enabling an external scheduler to respond to *inter-query dynamism*. Second, it can adapt to *intra-query data dynamism* within individual queries by choosing the best candidate plan with respect to varying data input. In §5.1, we first introduce the problem of **Speculative Adaptive Query Processing (SpecAQP)** and the system architecture that resolves both challenges with a unified abstraction of **Subquery Execution Unit (SEU)**. In §5.2, we discuss the details in SEU, the lowest-level preemptible units in the system. Finally, we demonstrate the framework enabling a Shortest Remaining Time First (SRTF) policy in §5.3.

5.1 Speculative Inter-Query AQP

While recent works [25, 38, 44] actively attempt to adopt on-GPU ML/LLM operators into query processing, they largely overlook

Algorithm 4 Pushdown across Source Partitions

```
function PdAcrossSrcPtt(Partition P)
    op_ptr ← P.src.parent
    seeds ← {P}
    while op_ptr ∉ {⋈, stateful ops} do
        if CanSwap(op_ptr, P.src) then
            P ← Swap(op_ptr, P.src)
            seeds ← seeds ∪ {P}
            op_ptr ← P.src.parent
        else
            seeds ← seeds ∪ {P}
            break
    return seeds
```

Algorithm 5 SpecAQP

```
Input: Scheduler scheduler, Profiler profiler
function InitQuery(q)
    q.subq_grp_tree ← GroupSubquery(q.candidate_plans)
    physical_ops ← InitAllPhysicalOps(q)
    Add(scheduler.queries, q)
    return physical_ops

function Execute()
    while scheduler.queries ≠ ∅ do
        cur_q ← Step(scheduler)
        cur_subq_grp ← GetReadySubq(cur_q.subq_grp_tree)
        repr_subq ← GetProfileSubq(cur_subq_grp)
        driver ← GetDriver(repr_subq)
        profile ← Run(driver, repr_subq, physical_ops)
        for subq ∈ cur_subq_grp.subqueries do
            PopulateProfile(subq, profile)
        best_subq ← GetBestSubq(cur_subq_grp)
        driver ← GetDriver(best_subq)
        result, stats ← Run(driver, best_subq, physical_ops)
        Update(scheduler, result, stats)
```

system-level integration with the model-serving backend. The gap makes performances highly dependent on hardware-scaling choices and therefore impractical to real-world scenarios. Aero [26] studies adaptive solutions on analytical queries over video databases with respect to GPU-side optimizations, but still assumes uninterrupted execution of individual queries with exclusive GPU occupancy.

Handling concurrent arrival and execution of queries is non-trivial mainly due to two complexities. First, the orchestration inside individual query's execution is often sophisticated (e.g., [26] applies a specialized implementation of Eddies [9] to only resolve filter reordering). Second, the presence of stateful operators (e.g., aggregation requires access to all possible data before producing any output) blocks streaming execution and requires additional state-preserving and recovering.

Subquery Groups SpecAQP tackles both challenges via disaggregated execution over *subqueries*. Subqueries are defined as logical subtrees rooted at a single blocking operator with all descendant nodes being non-blocking (e.g., filters, joins). A subquery is the maximal unit that allows non-blocking operators to stream batches to the root and safely yield execution upon preemption. The root operator can preserve its state simply by buffering the incoming batches upon preemption, incurring no extra memory overhead. Once all batches are buffered, the root evaluates the blocking operation and materializes the result for downstream use.

Subqueries are preemptible at batch-level granularity. Consequently, the original logical plan can be decomposed into an n-ary tree hierarchy (i.e. *subquery trees*) of these preemptible subqueries by partitioning on blocking operators. As discussed in §4.1, blocking operators are strictly non-commutative, serving as rigid boundaries against the relocation of other operators. Because of this structural constraint, the equivalent plans enumerated in §4.2 share an isomorphic subquery tree topology. Consequently, each topological slot maps to a localized space of equivalent subquery variants from the plans, i.e. *subquery groups*. The subqueries in a group share the same set of local operators.

Subquery Execution Unit (SEU) Subquery group seamlessly supports SpecAQP's disaggregated execution. It enables intra-query AQP where a cost model evaluates subquery variants per batch and picks the best one to execute. It also supports inter-query dynamism, allowing execution to be preempted and later resumed using any equivalent subquery in the group. To manage this dual-level flexibility, SpecAQP assigns each group an SEU, which we detail in §5.2.

Workflow Figure 4 illustrates the architecture of SpecAQP in a complete scheduler iteration during execution. The scheduler is responsible of keeping track of all query requests from users and deciding the query to be executed in the next iteration. The whole workflow consists of the following four steps.

(1) Select the ready subquery group. After the scheduler has selected the current query through its scheduling algorithm, we identify the subquery group in it that is *ready* to be executed. This is computed by traversing the n-ary tree of groups in depth-first order to find the rightmost subquery group that is not completed yet with all its children groups marked as completed. Once the *ready* group is found, we create an SEU for it.

(2) SEU profiles subquery group with SLM. Before an SEU starts execution of a subquery group, it needs to determine which subquery variant in it is the cheapest without causing serious execution blocking. Inspired by prior works on speculative inference [34, 40] and proxy approximation [28, 29], we run a *representative subquery* for the next batch using the physical operators on a speculative small language model (SLM). The SLM is efficient and incurs minimal cost compared to the LLM used for execution, but it offers an estimate for per-operator cost and statistics (e.g., token counts, latency, fan-in/out). SEU then populates all subqueries in the group, computing their costs using the operator profiles.

(3) SEU executes subquery group with LLM. Once all subqueries' costs can be determined, the SEU selects the cheapest one to be executed on the LLM, which fetches exactly one batch, processes it and suspends. If this is the last batch, the SEU marks the current group as completed and evaluates the root operator to yield the final result. It then creates a mock scan operator upon the materialized result, and inserts it into the downstream subquery group as the source of input.

(4) Update scheduler state. After execution over one batch is completed, the information used in the scheduling algorithm is updated with the current execution statistics, and the scheduler makes the decision for the next iteration.

5.2 Subquery Execution Unit

A Subquery Execution Unit (SEU) manages the execution of a single data batch once the scheduler activates its corresponding subquery group. The SEU serves as the basic granularity for cost profiling and execution in the adaptive framework. Upon instantiation by the external scheduler, the SEU retrieves the subquery group's operator pool. A driver program then links these operators into a physical subquery plan, streaming batch inputs from scan operators or upstream subquery yields through the plan in a depth-first order. To facilitate both speculative execution and actual processing, the SEU interfaces with both the SLM and the LLM. During cost estimation, it employs *Speculative Profiling*, where it streams a single batch through the SLM to materialize the physical costs of each operator. During actual execution, the SEU processes the batch identically, but routes it through the LLM.

Populating Costs While runtime profiling captures highly accurate execution metrics, exhaustively profiling the exponential search space of equivalent operator topology within a subquery group is prohibitively expensive even with the SLM. To address this, the SEU profiles only a single representative plan topology (*repr_subq* in line 11 of Alg.5), and derives the costs of all alternative equivalent plans via a virtual cost evaluator. We select the topology with the maximum pushdown of row-level operators (e.g., filters and projections) as the representative plan. During speculative profiling, each operator evaluates the *full batch* instead of the rows that pass the previous filtering. The SEU records per-row outcomes of filters and joins as bitmaps and faithfully evaluates the bitwise operations to estimate the intermediate cardinalities of each candidate plan. The evaluator then aggregates the corresponding operator costs according to each candidates' topology. We detail the profiler design in Appendix B [6]. After doing so, within the SEU, the topology with lowest cost and execute its physical plan is picked, as described in line 13-16 of Alg.5. Notably, this full-batch profiling can lead to substantially more work as it forgoes reduction of input cardinality from upstream operators. Ideally, using a fast yet performative SLM helps mitigate this profiling overhead. To further control profiling cost, SeqoFlow supports both periodic profiling and profiling triggered by a large mismatch between estimated and observed execution costs.

5.3 SRTF Scheduling

Shortest Remaining Time First (SRTF) scheduling is a simple yet theoretically optimal algorithm to minimize latency when serving ad hoc arrival requests. It reduces the cumulative queue waiting time of each request by always serving the one with the shortest remaining time to completion first. In most existing frameworks like LOTUS, query execution is non-preemptive, and once a complex physical query plan is dispatched, it must run to completion. This creates severe head-of-line blocking, where newly arrived and short queries are forced to wait for long-running, active queries to finish. Our system overcomes this limitation by enabling batch-level preemption mid-flight. Because the SEU processes data and materializes costs at the granularity of individual batches, the scheduler can possess an accurate and dynamically updated estimate of the remaining execution time for all active subquery groups (line 18 in Alg.5). This fine-grained, cost-aware context switching allows our system to natively support SRTF.

6 IMPLEMENTATION

We built SeqoFlow with 6K lines of Python code. We implement the semantic operators following [44] and adopt Pandas DataFrame [43] as the base relational data model. In this section, we discuss several implementation details.

Materialized Nested-Loop Join. Streaming joins such as the Symmetric Hash Join [57] and Ripple Join [20] maintain symmetric state for both inputs and require complex eviction policies at scale. To support batch-level yielding, we instead implement a materialized nested-loop join [19, 51], consistent with existing semantic join implementations [44, 55]. It caches only the active outer batch while progressively materializing the inner relation, so a preempted join resumes without refetching its inner input.

Caching, Broadcasting, and Fingerprinting. Mid-flight plan changes alter the state buffered inside joins: once an operator moves across a join, the cached inner relation cannot be replayed blindly without redundant or missing semantic processing. Each batch therefore carries lineage metadata maintained by three primitives. *Caching* buffers per-batch results of row-wise operators for reuse. *Broadcasting* evaluates an operator only on rows with distinct values in its input columns (e.g., a filter's predicate key) and copies the results to duplicate rows. *Fingerprinting* records the set of operators each batch has visited, similar to Eddies [9].

Scheduler Cost Modeling The execution cost of each operator is formulated as a weighted sum of its prefill and decode tokens. To support SRTF scheduling, the system maps this cost to remaining time using dynamically profiled scaling factors. Specifically, at scheduling step k and wall time t, SRTF selects $\text{argmin}_i\ \text{score}_i(t)$-th query to be executed in the next step. The score is defined by:

$$r_{i,k} = \xi \cdot (\gamma_1 \cdot \text{prefill} + \gamma_2 \cdot \text{decode} + \gamma_3), \quad \phi_k = \mathbb{1}[k > 0]$$
$$\hat{R}_{i,k} = \phi_k \Big[(1-\beta)\left(\hat{R}_{i,k-1} - (s_{i,k} - s_{i,k-1})\right) \Big] + \beta^{\phi_k} r_{i,k}$$
$$\text{score}_i(t) = \hat{R}_i(t) - \alpha\big(t - a_i - s_i(t)\big)$$

$r_{i,k}$ estimates token costs. $\hat{R}_{i,k}$ models actual remaining work by applying a residual smoothing factor β. $s_{i,k}$ or $s_i(t)$ is the total attained service of the i-th query at t (step k), a_i is the arrival time. To prevent starvation of long-running queries in SRTF, we apply an aging factor α penalizing long waits with deficient attained service. After step k, only ξ is updated to match newly completed work $\hat{R}_{i,k-1} - r_{i,k}$ with $s_{i,k} - s_{i,k-1}$, i.e. the newly attained service. This online calibration allows remaining-time estimates to reflect the backend's observed execution performance, incorporating changes in inference conditions into subsequent scheduling decisions.

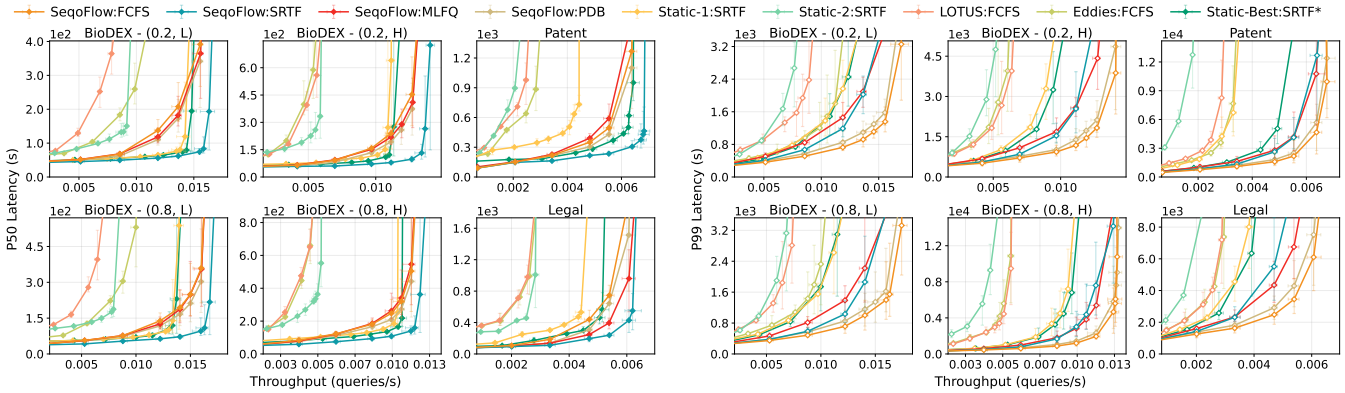


Figure 5: Latency vs Throughput in continuous arrivals – P50 and P99 of completed queries after certain cutoff in continuous runs. Runtime settings are denoted as {Method : Scheduler}. Geometric means across 10 runs are reported with min–max span.

7 EVALUATION

7.1 Experiment Setup

Workload Generation To our knowledge, no existing benchmark stress-tests the runtime dynamism in semantic queries. We therefore develop a configurable testbed that generates diverse settings. We utilize BioDEX [18], a semantically rich, high-dimensional biomedical dataset to construct controlled, representative and holistic evaluation. First, 9 representative query templates from the TPC-H [15] benchmark covering a wide range of common patterns in plan topology are selected as base plans, while we replace parts of their regular operators with LLM-based operators. We then configure these workloads with 2 parameters: the **variances of selectivities** $\delta(\sigma_{op})$ and **sequence lengths** $\delta(seq_len)$ among individual operators. Additionally, the selectivity of the entire query $\mathcal{S}$ is used to control the feasible space of the two parameters.

For input (or prefill) sequence lengths, we partition the input space into 3 tiers: **H**igh, **M**edium and **L**ow $\delta(seq_len)$. A workload is constructed by sampling sequence lengths uniformly, with 50% of operators drawn from the lower interval and 50% from the upper interval of the selected tier. Then, given $\delta(\sigma_{op})$, the selectivities of selective operator (filters and joins) are randomly generated by the testbed to produce a target end-to-end query selectivity $\mathcal{S}$. We consider (1) $\delta(\sigma_{op}) = 0.2$ and (2) $\delta(\sigma_{op}) = 0.8$, reflecting queries with more or less discrepant selectivity ranges respectively. In the following sections, unless otherwise noted, controlled settings are denoted as $(\delta(\sigma_{op}), \delta(seq_len))$[3].

To test the system against real-world scenarios, we extend the selected query templates to 2 additional corpora with only the definition (i.e., instructions to the LLM) changed within each operator, namely Legal [13] and Patent [1]. However, in these workloads, no configuration is attached. The cost and selectivity of each operator are determined solely by the actual semantics.

Baselines We compare system performance with the following **equivalence-preserving** baselines for comparison.

LOTUS [44]. This baseline features a declarative system with predefined semantic operators. To isolate the performance of query execution and scheduling, we disable LOTUS's model cascading feature, ensuring a strictly lossless execution using the identical underlying LLM. For query evaluation, the LOTUS baseline executes the default static logical topology provided by the initial query inputs, as it does not natively support mid-flight plan optimization.

Eddies [9]. This approach proposes an Eddy operator, a runtime router that forwards data to different operators according to their respective cost statistics. It allows adaptively reordering the operators during execution. We implement Eddies on top of LOTUS with additional support for batch-by-batch processing. Since the architecture does not consider operator dependencies, we manually bind operators with dependency constraints into fused operators, and leverage existing approaches [10, 12, 26, 47] to decide the score of each operator as $cost/(1 - selectivity)$ for routing.

Static-1/2. We implement these baselines in SeqoFlow by applying rule-based optimizations adopted from [39, 54] and human-curated operator-ordering heuristics [22] to reorder operators. We use these plans as strong, workload-specific heuristic baselines. Specifically, Static-1 uses predicate pushdown and Static-2 pushes semantic operators up to the root of the plan.

Static-Best*. This baseline executes a single fixed plan throughout, namely the best plan in hindsight among all enumerated candidates, or the first plan selected by SeqoFlow when the plan space is too large to execute exhaustively.

All baseline implementations adopt related techniques mentioned in §6 to ensure a fair comparison. We use Qwen3-4B and Qwen3-32B [60] for speculative profiling and execution, respectively. We serve them on two separate vLLM [33] instances colocated on an 8×A100 NVIDIA GPU server with tensor parallelism (TP=8), and enable chunked prefill [8] with ≤4096 tokens per chunk. SeqoFlow issues LLM calls at the query level and makes no assumption about how the LLM is deployed.

We include profiling overhead throughout the reported results.

7.2 End-to-end Performance

7.2.1 Effectiveness of Inter-Query Scheduling. Figure 5 compares SeqoFlow with run-to-completion baselines under continuous arrivals, using several scheduling policies implemented in SeqoFlow. Randomized queries arrive as a Poisson process with rate λ =

[3]We present results under High and Low settings due to space constraints. For all experiments in §7 without specifying testbed configurations, we use BioDEX-(0.2, H).

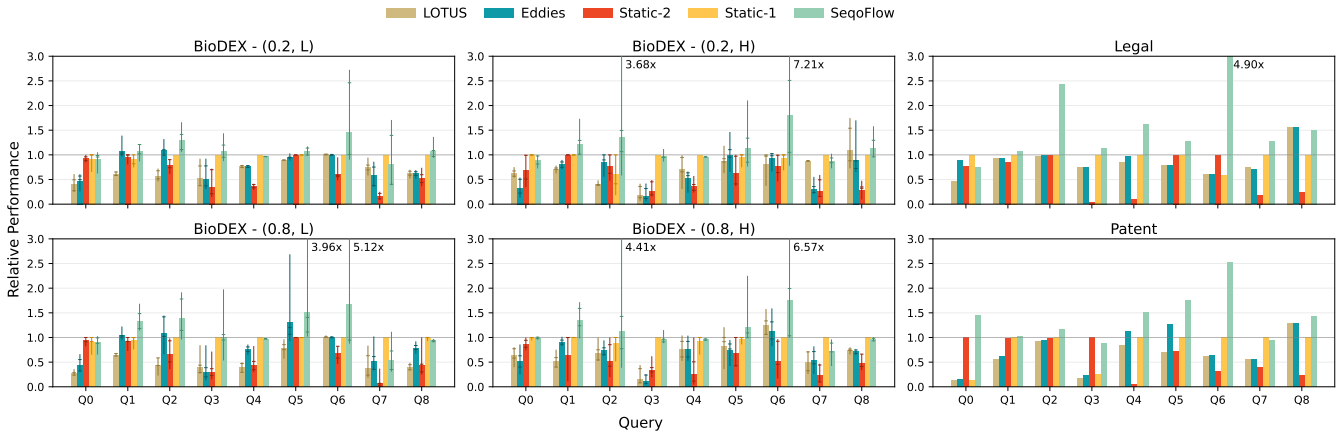


Figure 6: Single-query optimization – SeqoFlow can optimize a single query via plan enumeration and adaptively selecting the best plan at runtime. Relative performance is calculated as $min(Static1, Static2)/latency$, i.e. higher is better. Geometric means across 10 runs are reported with min–max span and 25th/75th percentiles on whiskers.

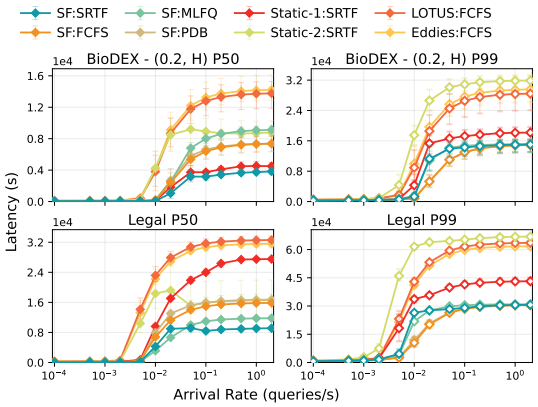


Figure 7: Finite arrivals with varying arrival rates – SF stands for SeqoFlow.

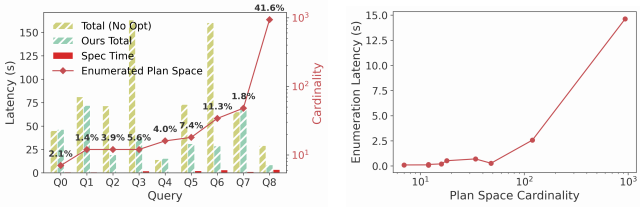
$load/E[latency]$, where $E[latency]$ is the mean latency of queries run in isolation across all approaches and random seeds. After a fixed-time warmup, 1,000 queries arrive in a sufficient window as the main cohort. The final completion window then measures the whole cohort to complete while allowing new arrivals. Results therefore exhibit a 1,000-job stream in the middle of a continuous arrival. Since SeqoFlow supports most preemption-based schedulers, we also implement **PreemptDB (*PDB*)** [23]'s policy for comparison.

SeqoFlow boosts system throughput at fixed latency budget as well as system capacity. Under the same SRTF scheduler, SeqoFlow sustains more load at a given latency and has higher capacity than static plans. With the scheduler held fixed at SRTF, SeqoFlow sustains up to 1.9× higher throughput than Static-1 and up to 4.3× higher than Static-2 at the same P50 latency, and it saturates at a strictly higher load on every workload. Saturation of system capacity can be solely attributed to per-query execution cost, as the scheduler cannot change the total work the system must perform. SeqoFlow also exceeds Static-Best*, which picks the best fixed plan per workload with hindsight. No single static plan, even one chosen by an oracle, reaches the capacity that SeqoFlow attains by adapting its plan at runtime, and the advantage is largest on Patent and Legal, where operator costs vary most across inputs.

SRTF contributes to lower P50 latencies at varying loads. Within SeqoFlow, SRTF reduces P50 latency by 2.5–4× compared to FCFS, MLFQ, and PreemptDB near saturation, while matching them at low load. This isolates the scheduler's contribution. With the plan held fixed, a scheduling policy can only reorder queued work. Its benefit gradually grows with queueing delay as load increases. As load approaches system capacity, FCFS, MLFQ, and PDB rise sharply, while SRTF is more resistant in queuing inflation. This behavior matched the theoretical expectation of SRTF minimizing mean response time, as serving short queries first keeps most queries from waiting even close to saturation.

SRTF supported by SeqoFlow surpasses run-to-completion baselines in P50 and P99 latencies. Although SRTF is known for inflating tail latency by starving long jobs, SeqoFlow with SRTF sustains 1.9× more load than LOTUS and Eddies with FCFS before P99 latency inflates. It also stays within 1.4× of SeqoFlow with FCFS. First, as SeqoFlow reduces per-query service time, it lowers the entire latency distribution, including the tail. Second, SeqoFlow's SRTF ages waiting queries (§6), which bounds how long a long query can be deferred and limits the queueing delay that starvation would otherwise add to the tail. Selecting efficient plans combined with bounded deferral thus outperforms the baselines, even though they use FCFS which prefers reducing tail latency.

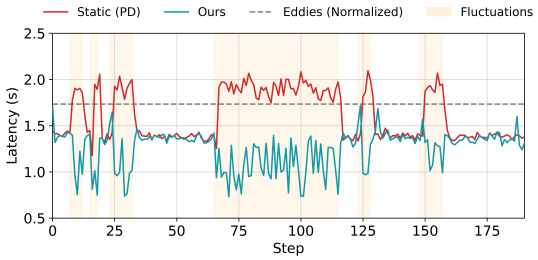
SeqoFlow speeds up job completions in finite workloads. Figure 7 shows latency against offered arrival rate in a simulated run with 200 arrivals of real traces collected from individual queries. Beyond saturation, latency plateaus because each run issues a finite query stream, SeqoFlow with SRTF achieves the lowest P50 latency on both workloads, which is 3.6× and 3.5× lower on both datasets than LOTUS with FCFS. SeqoFlow's P99 plateau is also 2× below LOTUS. This confirms that even under sustained overload, aging keeps SRTF's tail latency comparable to FCFS.



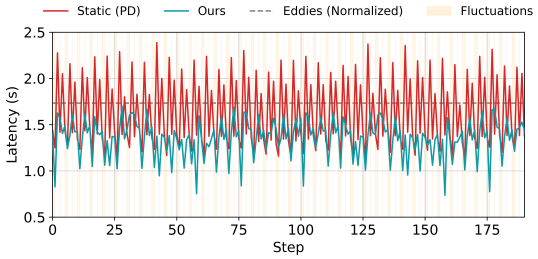
(a) Numbers indicate the percentage of E2E system latency attributed to speculative profiling overhead.

(b) The overhead of static plan enumeration with different equivalent plan search space cardinality.

Figure 8: Latency breakdown in SeqoFlow



(a) Random continuous fluctuation



(b) Highly dynamic fluctuation

Figure 9: Adaptiveness to fluctuations in workloads – SeqoFlow consistently converges to lower-cost candidate plans when fluctuation happens. Each step processes one batch.

7.2.2 Single-Query Optimization. We further validate SeqoFlow in terms of automatically deriving a plan with near-optimal performance when data characteristics are relatively controlled during runtime. As illustrated in Figure 6, SeqoFlow consistently outperforms LOTUS across all evaluated settings, achieving speedups of up to 7×. Crucially, it reliably matches the best performance of the human-curated baselines (Static-1 and Static-2) and even surpasses them in several configurations. This reveals that plan enumeration (§4) discovers optimization opportunities that rigid static rules miss, while our cost-based selection (§5.2) correctly picks a competitive topology. Furthermore, for complex plans with highly variable semantic execution costs, SeqoFlow manifests a promising advantage over simple and localized data-routing like Eddies. Eddies also lacks native support for inter-operator data dependencies and restricts reordering to a limited operator subset, making it struggle to optimize such settings.

7.2.3 Latency Breakdown. We further analyze the runtime overhead introduced by SeqoFlow, including plan enumeration and speculative profiling. As both components depend on the cardinality of the equivalent plan space, we evaluate their overhead under varying search space sizes in Figure 8. Figure 8a shows that speculative profiling introduces less than 10% overhead for most queries. Similarly, Figure 8b demonstrates that plan enumeration contributes negligible compilation overhead in typical cases. Even for extreme cases with plan spaces containing over 1,000 equivalent plans, the optimization overhead remains justified by the resulting execution time reduction. For the most challenging case (Q8), speculative profiling accounts for 41.6% of execution time. However, SeqoFlow still achieves substantial performance improvements compared to executing without optimization. This efficiency is enabled by SeqoFlow's compact plan enumeration strategy and lightweight profiling mechanism, which avoid exhaustive evaluation of all equivalent plans while preserving accurate cost estimation.

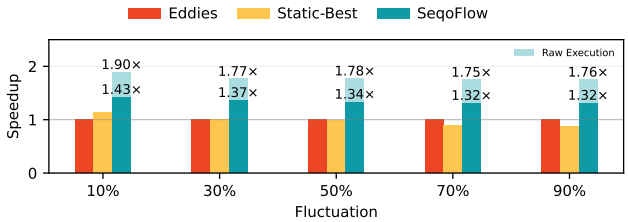


Figure 10: Speedup over Eddies under fluctuations.

7.3 Adaptiveness to Intra-Query Dynamism

To further test SeqoFlow's adaptiveness to dynamism, the costs and selectivities of selected operators are flipped in a portion (10%–90%) of batches. For instance, a filter that is originally cheap and selective becomes expensive and non-selective, so the plan that is best for the first batch loses its advantage on the flipped batches. Figure 9 demonstrates the per-batch latency of a representative query under both random continuous and highly dynamic fluctuation. SeqoFlow switches to a cheaper plan and returns once the flip in the injected fluctuation windows. In Figure 10, as the flipped portion grows, Static-Best* steadily degrades and falls below Eddies, since no fixed plan remains optimal once operator costs shift across batches. SeqoFlow re-selects plans batch by batch and outperforms both at every portion, achieving 1.32–1.43× speedups over Eddies end to end. Meanwhile, we show that SeqoFlow can be more effective when only considering execution cost (*Raw Execution*). This means SeqoFlow's sensitivity to runtime dynamism can be restrained by the SLM when cost drifts are detected frequently, triggering more profiling dispatched to capture the latest costs.

7.4 Validation of Cost Modeling

SeqoFlow relies on speculative cost estimates both to select plans within an SEU (§5.2) and to prioritize queries under SRTF (§5.3). We validate the estimates against each decision.

Speculative profiling on the SLM reliably selects the best plans for the LLM. Plan selection consumes the estimated cost of each candidate, which compounds operator selectivities with token counts and per-call latency. We therefore measure the relative error between the predicted cost from SLM profiling and the actual cost

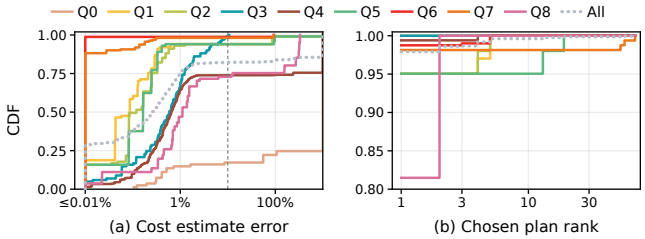


Figure 11: Accuracy of speculative cost estimation – (a) Relative cost error of each batch, with the dashed line at 10%. (b) Rank of the chosen plan among all candidates by actual cost.

Table 1: Ranking quality of estimated against actual remaining time across active queries under 90% input load.

	Spearman ρ		Acc_{Pair}	
SLM/LLM	w/o Age	Age	w/o Age	Age
4B/32B	0.748	0.714	84.6%	74.8%
4B/DeepSeek-Flash	0.897	0.705	93.9%	88.2%
4B/gpt-6-luna	0.923	0.755	94.5%	94.1%

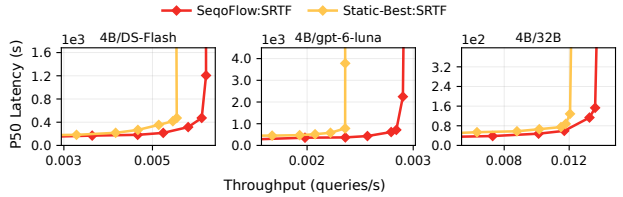


Figure 12: End-to-end performance across SLM/LLM combinations under SRTF.

on the LLM, over all batches of all queries with 70% of batches flipped (§7.3). Over 80% of batches are estimated within 10% error (Figure 11a). For over 95% of batches, SeqoFlow executes one of the top-2 plans under actual costs (Figure 11b).

Speculative estimates rank concurrent queries accurately for SRTF. SRTF depends only on the relative order of remaining times, which SeqoFlow preserves with high fidelity. Without aging, SeqoFlow orders up to 94.5% of query pairs correctly, with Spearman ρ up to 0.92 (Table 1). Aging deliberately trades part of this order for bounded waiting, yet keeps at least 74.8% of pairs correctly ordered.

Together, accurate plan selection and query ranking deliver consistent end-to-end gains across models. Pairing the same 4B SLM with official APIs of DeepSeek-Flash [17], and GPT-6 Luna, SeqoFlow sustains higher throughput than Static-Best* under every combination, whether the LLM is self-hosted or served through an API (Figure 12). Since Static-Best* fixes the best plan in hindsight under the same scheduler, this advantage stems from per-batch plan selection guided by speculative estimates.

7.5 Case Studies

Figure 6 has shown that the benefit of SeqoFlow varies across different workloads. We present a brief study on SeqoFlow's best and worst performances. In the bad cases, Q0 has only 4 operators with few admissible candidates, and it primarily tests whether plan transformations preserve the query result. It therefore leaves little headroom for improvement from discovering the best plan candidate, while speculative profiling can also add overhead. In contrast, Q1 and Q6 contain multiple semantic filters, projections and joins, yielding more admissible plan candidates with respect to dependency constraints. An early selective filter can avoid downstream projection calls, whereas a costly filter that retains most rows may be better delayed. The favorable order depends on runtime selectivity and token cost, which fixed pushdown or pushup rules cannot determine in advance, making them highly profitable. SeqoFlow uses its runtime estimates to select among these plans, leading to larger gains on these settings.

8 DISCUSSION AND FUTURE WORK

Concurrency Control. SeqoFlow naturally supports concurrent OLAP workloads by isolating memory spaces of relevant data structures and cache buffers. However, extending the system to handle concurrent data modifications and associated consistency challenges remains an interesting direction.

Search Space Pruning. SeqoFlow captures equivalent logical plans to adapt to runtime operator behavior. As with exhaustive plan enumeration, the search space can grow combinatorially. In the extreme case of fully independent operators, the number of equivalent orderings can reach $O(N!)$. Developing principled pruning techniques that preserve runtime adaptivity while reducing exploration overhead is an important trade-off for future work. With exact costs, exhaustive enumeration guarantees the best plan within the supported space, but finding that plan in practice need not require retaining every candidate. Besides the well-known pruning heuristics in §4.2, advanced runtime pruning could discard plans that are observably poor on current batches and retain promising ones, revisiting them if costs shift.

System Integration. SeqoFlow is designed for storage backends with batch support as discussed in §3. It is compatible with existing physical optimizations and orthogonal to the design spaces of LLM serving systems and operator implementations.

Cross-query computation reuse. Although SeqoFlow improves efficiency in serving multiple semantic queries concurrently, it does not assume any shared operators across different queries as well as the actual LLM calls from them. Exploiting shared work among concurrent queries is previously covered by works such as [11, 50], and can be further addressed as a materialized view problem [21] in the multi-query setting.

9 CONCLUSION

In this work, we present SeqoFlow, a unified framework that jointly addresses different types of runtime dynamism. By modeling operator constraints and equivalence boundaries, enabling fine-grained, structure-aware execution, and introducing lightweight speculative cost estimation, our approach effectively adapts to dynamic workloads. Experimental results show that SeqoFlow significantly outperforms existing methods. Overall, this work highlights the importance of co-designing optimization and scheduling for AI-powered data systems and provides a foundation for future semantic query processing engines.

REFERENCES

[1] [n.d.]. allenai/us-patents · Datasets at Hugging Face. https://huggingface.co/datasets/allenai/us-patents

[2] [n.d.]. Data Types and In-Memory Data Model — Apache Arrow v25.0.0. https://arrow.apache.org/docs/python/data.html

[3] [n.d.]. Enrich data using AI Functions. https://docs.databricks.com/aws/en/large-language-models/ai-functions

[4] [n.d.]. Machine learning and AI functions in GoogleSQL. https://docs.cloud.google.com/spanner/docs/reference/standard-sql/ml-functions

[5] [n.d.]. pyspark.sql.DataFrame. https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.html

[6] [n.d.]. Seurgul/SeqoFlow: Navigating Runtime Dynamism in Semantic Query Serving. https://github.com/Seurgul/SeqoFlow/tree/report

[7] [n.d.]. SQL reimagined for the AI era with BigQuery AI functions. https://cloud.google.com/blog/products/data-analytics/sql-reimagined-for-the-ai-era-with-bigquery-ai-functions

[8] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.

[9] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: continuously adaptive query processing. In *SIGMOD 2000*. 261–272. https://doi.org/10.1145/342009.335420

[10] Shivnath Babu, Pedro Bizarro, and David DeWitt. 2005. Proactive re-optimization. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data* (Baltimore, Maryland) *(SIGMOD '05)*. Association for Computing Machinery, New York, NY, USA, 107–118. https://doi.org/10.1145/1066157.1066171

[11] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, Liling Tan, Dmitrijs Milajevs, Geeticka Chauhan, Jeremy Gwinnup, and Elijah Rippeth (Eds.). Association for Computational Linguistics, Singapore, 212–218. https://doi.org/10.18653/v1/2023.nlposs-1.24

[12] Pedro Bizarro, Shivnath Babu, David DeWitt, and Jennifer Widom. 2005. Content-based routing: different plans for different data. In *Proceedings of the 31st International Conference on Very Large Data Bases* (Trondheim, Norway) *(VLDB '05)*. VLDB Endowment, 757–768.

[13] Umar Butler. 2025. Open Australian Legal Corpus. https://doi.org/10.57967/hf/2784

[14] Surajit Chaudhuri and Kyuseok Shim. 1999. Optimization of queries with user-defined predicates. *ACM Transactions on Database Systems (TODS)* 24, 2 (1999), 177–228.

[15] Transaction Processing Performance Council. 2026. TPC-H Benchmark. https://www.tpc.org/tpch/

[16] Hanjun Dai, Bethany Y Wang, Xingchen Wan, Bo Dai, Sherry Yang, Azade Nova, Pengcheng Yin, Phitchaya M Phothilimthana, Charles Sutton, and Dale Schuurmans. 2024. Uqe: A query engine for unstructured databases. *Advances in Neural Information Processing Systems* 37 (2024), 29807–29838.

[17] DeepSeek-AI. 2026. DeepSeek-V4.1-Flash Technical Report. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/blob/main/DeepSeek_V41_Tech_Report.pdf.

[18] Karel D'Oosterlinck, François Remy, Johannes Deleu, Thomas Demeester, Chris Develder, Klim Zaporojets, Aneiss Ghodsi, Simon Ellershaw, Jack Collins, and Christopher Potts. 2023. BioDEX: Large-Scale Biomedical Adverse Drug Event Extraction for Real-World Pharmacovigilance. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 13425–13454. https://doi.org/10.18653/v1/2023.findings-emnlp.896

[19] Goetz Graefe. 1994. Volcano - an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering* 6, 1 (1994), 120–135.

[20] Peter J Haas and Joseph M Hellerstein. 1999. Ripple joins for online aggregation. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*. 287–298.

[21] Oktie Hassanzadeh and Dharmashankar Subramanian. 2026. Semantic Views: Certified Reuse of LLM Labels Across Semantic Queries. *arXiv preprint arXiv:2610.12768* (2026).

[22] Joseph M Hellerstein and Michael Stonebraker. 1993. Predicate migration: Optimizing queries with expensive predicates. In *SIGMOD 1993*. 267–276.

[23] Kaisong Huang, Jiatang Zhou, Zhuoyue Zhao, Dong Xie, and Tianzheng Wang. 2025. Low-Latency Transaction Scheduling via Userspace Interrupts: Why Wait or Yield When You Can Preempt? *Proc. ACM Manag. Data* 3, 3, Article 182 (June 2025), 25 pages. https://doi.org/10.1145/3725319

[24] Navin Kabra and David J DeWitt. 1998. Efficient mid-query re-optimization of sub-optimal query execution plans. In *Proceedings of the 1998 ACM SIGMOD international conference on Management of data*. 106–117.

[25] Gaurav Tarlok Kakkar, Jiashen Cao, Pramod Chunduri, Zhuangdi Xu, Suryatej Reddy Vyalla, Prashanth Dintyala, Anirudh Prabakaran, Jaeho Bang, Aubhro Sengupta, Kaushik Ravichandran, Ishwarya Sivakumar, Aryan Rajoria, Ashmita Raju, Tushar Aggarwal, Abdullah Shah, Sanjana Garg, Shashank Suman, Myna Prasanna Kalluraya, Subrata Mitra, Ali Payani, Yao Lu, Umakishore Ramachandran, and Joy Arulraj. 2023. EVA: An End-to-End Exploratory Video Analytics System. In *Proceedings of the Seventh Workshop on Data Management for End-to-End Machine Learning* (Seattle, WA, USA) *(DEEM '23)*. Association for Computing Machinery, New York, NY, USA, Article 8, 5 pages. https://doi.org/10.1145/3595360.3595858

[26] Gaurav Tarlok Kakkar, Jiashen Cao, Aubhro Sengupta, Joy Arulraj, and Hyesoon Kim. 2025. Aero: Adaptive Query Processing of ML Queries. *Proc. ACM Manag. Data* 3, 3, Article 174 (June 2025), 27 pages. https://doi.org/10.1145/3725408

[27] Daniel Kang, Peter Bailis, and Matei Zaharia. 2018. Blazeit: Optimizing declarative aggregation and limit queries for neural network-based video analytics. *arXiv preprint arXiv:1805.01046* (2018).

[28] Daniel Kang, Edward Gan, Peter Bailis, Tatsunori Hashimoto, and Matei Zaharia. [n.d.]. Approximate Selection with Guarantees using Proxies. *Proceedings of the VLDB Endowment* 13, 11 ([n. d.]).

[29] Daniel Kang, John Guibas, Peter Bailis, Tatsunori Hashimoto, Yi Sun, and Matei Zaharia. 2021. Accelerating approximate aggregation queries with expensive predicates. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2341–2354.

[30] Konstantinos Karanasos, Matteo Interlandi, Doris Xin, Fotis Psallidas, Rathijit Sen, Kwanghyun Park, Ivan Popivanov, Supun Nakandal, Subru Krishnan, Markus Weimer, et al. 2019. Extending relational query processing with ML inference. *arXiv preprint arXiv:1911.00231* (2019).

[31] Fisnik Kastrati and Guido Moerkotte. 2016. Optimization of conjunctive predicates for main memory column stores. *Proceedings of the VLDB Endowment* 9, 12 (2016).

[32] Donald E Knuth. 1998. *The Art of Computer Programming: Sorting and Searching, volume 3*. Addison-Wesley Professional.

[33] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[34] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.

[35] Alon Y Levy, Inderpal Singh Mumick, and Yehoshua Sagiv. 1994. Query optimization by predicate move-around. In *VLDB*, Vol. 94. 12–15.

[36] Guoliang Li, Ji Sun, Lijie Xu, Shifu Li, Jiang Wang, and Wen Nie. 2024. Gaussml: An end-to-end in-database machine learning system. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 5198–5210.

[37] Paweł Liskowski, Benjamin Han, Paritosh Aggarwal, Bowei Chen, Boxin Jiang, Nitish Jindal, Zihan Li, Aaron Lin, Kyle Schmaus, Jay Tayade, Weicheng Zhao, Anupam Datta, Nathan Wiegand, and Dimitris Tsirogiannis. 2026. Cortex AISQL: A Production SQL Engine for Unstructured Data. In *Companion of the International Conference on Management of Data* (India) *(SIGMOD Companion '26)*. Association for Computing Machinery, New York, NY, USA, 400–412. https://doi.org/10.1145/3788853.3803093

[38] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpzest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*. 2.

[39] Qiuyang Mang, Yufan Xiang, Hangrui Zhou, Runyuan He, Jiaxiang Yu, Hanchen Li, Aditya Parameswaran, and Alvin Cheung. 2026. Horrila: Cost-Based Placement of Semantic Operators in Hybrid Query Plans. arXiv:2604.09944 [cs.DB] https://arxiv.org/abs/2604.09944

[40] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 932–949.

[41] Thomas Neumann, Sven Helmer, and Guido Moerkotte. 2005. On the optimal ordering of maps and selections under factorization. In *21st International Conference on Data Engineering (ICDE'05)*. IEEE, 490–501.

[42] Thomas Neumann, Sven Helmer, and Guido Moerkotte. 2006. On the optimal ordering of maps, selections, and joins under factorization. In *Proceedings of the 23rd British National Conference on Databases, Conference on Flexible and Efficient Information Handling* (Belfast, Northern Ireland, UK) *(BNCOD'06)*. Springer-Verlag, Berlin, Heidelberg, 115–126. https://doi.org/10.1007/11788911_10

[43] The pandas development team. 2020. *pandas-dev/pandas: Pandas*. https://doi.org/10.5281/zenodo.3509134

[44] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proc. VLDB Endow.* 18, 11 (July 2025), 4171–4184. https://doi.org/10.14778/3749646.3749685

[45] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference

using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.

[46] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. 2024. Power-aware deep learning model serving with {μ-Serve}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 75–93.

[47] Francisco Romero, Johann Hauswald, Aditi Partap, Daniel Kang, Matei Zaharia, and Christos Kozyrakis. 2022. Optimizing Video Analytics with Declarative Model Relationships. *Proc. VLDB Endow.* 16, 3 (Nov. 2022), 447–460. https://doi.org/10.14778/3570690.3570695

[48] Matthew Russo, Tatsunori Hashimoto, Daniel Kang, Yi Sun, and Matei Zaharia. 2023. Accelerating Aggregation Queries on Unstructured Streams of Data. *Proceedings of the VLDB Endowment* 16, 11 (2023), 2897–2910.

[49] Matthew Russo, Chunwei Liu, Sivaprasad Sudhir, Gerardo Vitagliano, Michael Cafarella, Tim Kraska, and Samuel Madden. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *arXiv preprint arXiv:2505.14661* (2025).

[50] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E Gonzalez. 2025. vcache: Verified semantic prompt caching. *arXiv preprint arXiv:2502.03771* (2025).

[51] P. Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. 23–34.

[52] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3035–3048.

[53] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 173–191.

[54] Ji Sun, Guoliang Li, Peiyao Zhou, Yihui Ma, Jingzhe Xu, and Yuan Li. 2025. Agenticdata: An agentic data analytics system for heterogeneous data. *arXiv preprint arXiv:2508.05002* (2025).

[55] Immanuel Trummer. 2025. Implementing Semantic Join Operators Efficiently. *arXiv preprint arXiv:2510.08489* (2025).

[56] Midhul Vuppalapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. 2020. Building an elastic query engine on disaggregated storage. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 449–462.

[57] Annita N Wilschut and Peter MG Apers. 1991. Dataflow query execution in a parallel main-memory environment. In *Proceedings of the First International Conference on Parallel and Distributed Information Systems (PDIS)*. IEEE, 68–77.

[58] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).

[59] Renzhi Wu, Pramod Chunduri, Ali Payani, Xu Chu, Joy Arulraj, and Kexin Rong. 2024. SketchQL: Video Moment Querying with a Visual Query Interface. *Proc. ACM Manag. Data* 2, 4, Article 204 (Sept. 2024), 27 pages. https://doi.org/10.1145/3677140

[60] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[61] Zhihui Yang, Zuozhi Wang, Yicong Huang, Yao Lu, Chen Li, and X Sean Wang. 2022. Optimizing machine learning inference queries with correlative proxy models. *arXiv preprint arXiv:2201.00309* (2022).

[62] Junyi Zhao, Huanchen Zhang, and Yihan Gao. 2023. Efficient query reoptimization with judicious subquery selections. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–26.

[63] Junhao Zhu, Lu Chen, Xiangyu Ke, Ziquan Fang, Tianyi Li, Yunjun Gao, and Chrisitian S Jensen. 2026. Beyond Relational: Semantic-Aware Multi-Modal Analytics with LLM-Native Query Optimization. *Proceedings of the ACM on Management of Data* 4, 1 (SIGMOD (2026), 1–25.

A PROOFS OF PLAN ENUMERATION PROPERTIES

Fix a plan T with operator set V, $|V| = N$, and J joins. Operators are labeled; read sets are nonempty and origin-qualified, so each admissible join crossing selects a unique input branch. Let $\mathcal{R}(T)$ be its closure under admissible swaps and $\mathcal{E}_0(T)$ its memoized plan set without pruning.

A.1 Plan Space

Proposition 1 (Plan Space). For the forest F defined below,

$$|\mathcal{R}(T)| \le \frac{N!}{2^J \prod_{v\in V} h_F(v)}.$$

Dependency poset. Write $u \prec_D v$ for transitive producer–consumer dependence and $u \prec_Q v$ when u is a proper descendant of v in plan Q. Let B contain the precedence constraints imposed by the fixed skeleton and noncrossable boundaries. Retaining these constraints from T gives

$$P = (V, \prec_P), \qquad \prec_P = \mathrm{tc}(\prec_D \cup B),$$

with $\prec_P \subseteq \prec_Q$ for every $Q \in \mathcal{R}(T)$, where tc denotes transitive closure. Let $H = \mathrm{Cov}(P)$ be its Hasse edges, directed from smaller to larger elements. For any poset A, write $\mathcal{L}(A)$ for its linear extensions and $e(A) = |\mathcal{L}(A)|$.

Uniform hook construction. Let $P = P_1 \sqcup \cdots \sqcup P_m$, where each component P_i has a tree Hasse diagram and a unique maximal or unique minimal element r_i. Dualize precisely the reverse-rooted components:

$$F_i = \begin{cases} P_i, & r_i \text{ is maximal,} \\ P_i^{op}, & r_i \text{ is minimal,} \end{cases} \qquad F = \bigsqcup_{i=1}^{m} F_i.$$

For a singleton, either choice agrees. Each F_i is a rooted tree with unique maximum r_i. Its hook is uniformly the subtree size

$$h_F(v) = |\{u \in P_i : v \text{ lies on the path from } u \text{ to } r_i\}|.$$

Equivalently, using reflexive orders,

$$h_F(v) = \begin{cases} |\{u : u \preceq_{P_i} v\}|, & r_i \text{ is maximal,} \\ |\{u : v \preceq_{P_i} u\}|, & r_i \text{ is minimal.} \end{cases}$$

Lemma 2 (Uniform hook formula). *With this construction,*

$$e(P) = e(F) = \frac{N!}{\prod_{v\in V} h_F(v)}. \tag{1}$$

Proof. Given $\lambda \in \mathcal{L}(P)$, reverse its subsequence on each dualized component, leaving the positions occupied by that component unchanged. This involution gives a bijection $\mathcal{L}(P) \longleftrightarrow \mathcal{L}(F)$. Let $n_i = |P_i|$. Applying Knuth's formula [32] to each rooted F_i and interleaving the components gives

$$\begin{aligned} e(P) = e(F) &= \binom{N}{n_1, \ldots, n_m} \prod_{i=1}^{m} e(F_i) \\ &= \frac{N!}{\prod_i n_i!} \prod_{i=1}^{m} \frac{n_i!}{\prod_{v\in P_i} h_F(v)} \\ &= \frac{N!}{\prod_{v\in V} h_F(v)}. \end{aligned}$$

□

Lemma 3 (Join multiplicity). *Every reachable plan Q with J joins satisfies $|\mathcal{L}(Q)| \ge 2^J$.*

Proof. Fix $\lambda \in \mathcal{L}(Q) \subseteq \mathcal{L}(P)$ and a join j with nonempty input subtrees Q_L, Q_R. Let λ_L, λ_R be the restrictions of λ to their operators. These branches are incomparable, so every shuffle preserving λ_L and λ_R represents the same join with unchanged inputs. In particular,

$$\lambda_L \lambda_R \ne \lambda_R \lambda_L.$$

Replace the branch subsequence of λ by either concatenation, choosing one different from the original subsequence and keeping all other positions unchanged. This gives a distinct member of $\mathcal{L}(Q)$ representing the same plan.

More generally, writing $a = |Q_L|$, $b = |Q_R|$, and $e(Q) = |\mathcal{L}(Q)|$,

$$e(Q_j) = \binom{a+b}{a} e(Q_L) e(Q_R) \ge 2 e(Q_L) e(Q_R).$$

A unary root leaves e unchanged. Induction over the plan tree therefore gives one factor of at least two per join, hence $e(Q) \ge 2^J$. □

Proof of Proposition 1. First, distinct reachable plans have disjoint linear-extension sets. Contracting row-wise paths leaves the same fixed skeleton in every plan; each row-wise operator's admissible positions lie on one path because join crossings select a unique branch. Thus $Q \ne Q'$ either orders two operators oppositely within a row-wise path, or places a row-wise operator on opposite sides of a fixed operator. In either case, some u, v satisfy

$$u \prec_Q v, \qquad v \prec_{Q'} u,$$

so $\mathcal{L}(Q) \cap \mathcal{L}(Q') = \varnothing$. Since $\mathcal{L}(Q) \subseteq \mathcal{L}(P)$, the preceding lemmas give

$$\begin{aligned} 2^J |\mathcal{R}(T)| &\le \sum_{Q\in\mathcal{R}(T)} |\mathcal{L}(Q)| \\ &= \left| \bigcup_{Q\in\mathcal{R}(T)} \mathcal{L}(Q) \right| \\ &\le e(P) = \frac{N!}{\prod_{v\in V} h_F(v)}. \end{aligned}$$

Dividing by 2^J proves $|\mathcal{R}(T)| \le \frac{N!}{2^J \prod_{v\in V} h_F(v)}$. □

A.2 MECE

Proposition 2 Every memoized plan is equivalent to T and memoized once, and $\mathcal{E}_0(T) = \mathcal{R}(T)$.

Memoization Algorithm 3 and Algorithm 4 use set operation $\bigcup$ to iteratively include newly enumerated partitions into a set of seeds while making each member in the set unique. Then it calls the Merge operation to overwrite the current position in the plan T with the new partition(s). It is obvious that, when the merged partitions are different, the resulting full plan T' after Merge is also different. In actual implementation, to prevent duplicates in plan enumeration, we leverage a compact position vector to flag whether the algorithms have generated the same partition before.

Position vector. Given an anchor join j and a labeled operator list $W = (\omega_1, \ldots, \omega_m)$ covering the local transformation. Number the

Algorithm 6 Cost Propagation

Require: $\{K_{op}, r_{op}\}_{op \in T}$ from the preceding SLM profiling

function PopulateProfile($T, \{K_{op}, r_{op}, Cache_{op}\}$)
  $C \leftarrow 0$
  $\alpha_{op} \leftarrow 1$ for all $op \in T$
  for all joins $j \in T$ **do**
    $(L_j, R_j) \leftarrow$ outer and inner source-batch counts, each at least 1
    $\alpha_{op} \leftarrow \alpha_{op}/R_j$ for $op \in outer(j)$
    $\alpha_{op} \leftarrow 0$ if the inner side is materialized; otherwise $\alpha_{op}/(L_j R_j)$, for $op \in inner(j)$
  for all operators op in T, children before parents **do**
    if op is a scan **then**
      $R_{op} \leftarrow K_{op}$
    else
      $I_{op} \leftarrow R_{op_L} \times R_{op_R}$ for a join; otherwise $R_{child(op)}$
      $R_{op} \leftarrow$ ApplyMask($I_{op}, K_{op}, Cache_{op}$)
      if op is semantic **then**
        $n_{op} \leftarrow |\pi_{op}(I_{op}) \setminus \mathrm{dom}(Cache_{op})|$
        $C \leftarrow C + \alpha_{op}\, r_{op}\, n_{op}$
  return C

three arms of j by 0 (above), 1 (left), and 2 (right). For each ω_i, let b_i be its arm and $r_i \geq 1$ its rank counting outward from j. Define

$$\mathbf{p}_j(P) = (p_1, \ldots, p_m), \qquad p_i = 3(r_i - 1) + b_i + 1.$$

Each entry uniquely determines its arm and rank; hence, for fixed j, W, $\mathbf{p}_j(P) = \mathbf{p}_j(P') \Leftrightarrow P = P'$.

Lemma 4 (Coverage). *Starting from the promoted seed, unpruned within- and across-partition pushdown visits every admissible partition configuration through its layer-by-layer decomposition, possibly repeatedly.*

Proof. For a target Q, let $\mathcal{V}_k$ be the states visited at layer k. Let $C_k(Q)$ require agreement with Q on processed chains and placement of each remaining operator in its highest unprocessed chain on its path to Q. Set

$$\mathcal{F}_k(Q) = \mathcal{V}_k \cap C_k(Q).$$

(i) Assume Q is missing. Since the promoted seed is compatible, there is a first failure:

$$\mathcal{F}_k(Q) \neq \varnothing, \qquad \mathcal{F}_{k+1}(Q) = \varnothing.$$

(ii) Choose $S \in \mathcal{F}_k(Q)$. In each current chain W, let D contain the operators placed below the join in Q and set $U = W \setminus D$. Validity of Q implies

$$x \in W,\ y \in D,\ x \prec_D y \implies x \in D.$$

(iii) Thus D is an order ideal. For a topological order π_D and $\pi_U = Q|_U$,

$$\pi_D \pi_U \in \text{ApplyPermute}(dep_grps).$$

(iv) Target validity makes every operator in D crossable, with producers crossing first. Across-partition pushdown emits the prefix $|D|$, fixing U and transferring D to its target branches. Cartesian products of these choices give

$$S' \in \mathcal{F}_{k+1}(Q),$$

contradicting (i). At terminal chains, $D = \varnothing$ and permutation alone fixes their order. Therefore no admissible Q is missing. □

Proof of Proposition 2. By Lemma 4, every partition contributes all admissible variants. Cartesian products and Merge assemble every compatible full plan and recursion covers all layers. Since every transformation is admissible,

$$\mathrm{supp}\left(Visited(T)\right) = \mathcal{R}(T).$$

The complete tuple of labeled partition position vectors uniquely determines the merged plan. Memoizing only unseen tuples retains exactly one copy of each visited plan. Hence

$$\mathcal{E}_0(T) = \mathcal{R}(T),$$

with every member stored once, establishing MECE. □

B SPECULATIVE PROFILING

We specify the propagation of costs in speculative profiling.

For subquery plan T, C is the estimated cost. Operator op has input and output tuples I_{op}, R_{op}, result mask K_{op}, cost per evaluation r_{op}, amortization factor α_{op}, and uncached evaluation count n_{op}. The evaluation-key map π_{op} uses relevant source-row identities for filters and projections, full tuple identities for joins or unknown dependencies. The cache $Cache_{op}$ maps these keys to filter decisions or projected values and is empty for uncached operators. Let $\mathrm{dom}(Cache_{op})$ be the cache key set. For join j, $L_j, R_j \geq 1$ count outer and inner source batches, $\mathrm{outer}(j), \mathrm{inner}(j)$ are its child subtrees, and op_L, op_R denote the two children of a join.

Algorithm 6 details the PopulateProfile function from line 15 of Algorithm 5, where the obtained full-batch masks of each operator is used to compute the exact cost of each candidate plan T. By applying intersection to the per-operator masks, we can compute costs with any ordering of the set of profiled operators.